LAPORAN PRAKTIKUM MAPREDUCE HADOOP DENGAN JAVA



Dosen Pengampu:

Ersha Aisyah Elfaiz, S.Si., M.T.I.

Disusun oleh:

Muhammad Anwar Ismail (24051204161)

Program Studi Teknik Informatika

Fakultas Teknik

Universitas Negeri Surabaya

2026

DAFTAR ISI

DAFTAR ISI ..... **i**
DAFTAR GAMBAR **1**
BAB I PENDAHULUAN **2**
1.1 Latar Belakang 2
1.2 Tujuan Praktikum 2
BAB II DASAR TEORI **3**
2.1 Konsep MapReduce, HDFS, dan YARN 3
BAB III PEMBAHASAN **4**
3.1 Hasil dan Tangkapan Layar 4
3.2 Penelusuran Proses Map ke Shuffle dan Sort ke Reduce 17
3.3 Perbedaan Komponen dalam Klaster 18
BAB IV KESIMPULAN **20**
4.1 Kesimpulan 20
LAMPIRAN A COMMAND UTAMA **21**
LAMPIRAN B PORT YANG DIGUNAKAN **25**

DAFTAR GAMBAR

Gambar 3.1.1 - Pindah ke direktori tugas Minggu ke-4.png 4
Gambar 3.1.2 - Jalankan klaster Hadoop + YARN.png 4
Gambar 3.1.3 - Verifikasi apakah keenam layanan sudah menyala.png 5
Gambar 3.1.4 - Verifikasi YARN Node.png 5
Gambar 3.1.5 - Verifikasi DataNode HDFS - Access denied for user root.png 6
Gambar 3.1.6 - Verifikasi DataNode HDFS_1.png 6
Gambar 3.1.7 - Verifikasi DataNode HDFS_2.png 7
Gambar 3.1.8 - Menyiapkan File Input di HDFS.png 7
Gambar 3.1.9 - Kompilasi Java (Membuat file JAR).png..... 8
Gambar 3.1.10 - Rangkaian proses kompilasi Java di dalam terminal Linux.png 9
Gambar 3.1.11 - Menjalankan MapReduce Job - Error_1.png..... 10
Gambar 3.1.12 - Menjalankan MapReduce Job - Error_2.png..... 10
Gambar 3.1.13 - 10. Matikan - Hidupkan - Hapus FOlder Output yang gagal.png 12
Gambar 3.1.14 – 11. Kirim ulang kode Java - Masuk ke dalam terminal Linux NameNode - Lakukan kompilasi 13
Gambar 3.1.15 - Jalankan Ulang MapReduce Job_1.png..... 14
Gambar 3.1.16 - Jalankan Ulang MapReduce Job_2.png..... 14
Gambar 3.1.17 - Membaca Hasil Akhir dari HDFS..... 15
Gambar 3.1.18 - Memeriksa Port YARN Web UI melalui Powershell.png ... 15
Gambar 3.1.19 - Memeriksa YARN Web UI_1.png 16
Gambar 3.1.20 - Memeriksa YARN Web UI_1.png 16

BAB I
PENDAHULUAN

1.1 Latar Belakang

Volume data yang terus bertambah setiap hari menuntut cara pengolahan yang tidak lagi bergantung pada satu mesin saja. Praktikum minggu sebelumnya sudah membahas HDFS sebagai lapisan penyimpanan yang memecah file besar menjadi beberapa block dan menyebarkannya ke sejumlah DataNode. Praktikum kali ini melanjutkan penyimpanan tersebut ke lapisan komputasi bernama MapReduce, yang memungkinkan proses pengolahan data dijalankan secara paralel di beberapa node melalui pengaturan YARN. Studi kasus WordCount dipilih karena polanya cukup sederhana untuk dipahami di tahap awal, namun tetap mencerminkan keseluruhan alur kerja Map, Shuffle & Sort, dan Reduce yang menjadi inti dari MapReduce. Klaster Hadoop 3.4.3 dibangun menggunakan Docker dengan konfigurasi satu NameNode, dua DataNode, satu ResourceManager, dan dua NodeManager, sehingga proses eksekusi job dapat diamati langsung pada lingkungan dengan node-node yang benar-benar terpisah di dalam container.

1.2 Tujuan Praktikum

- Memahami paradigma MapReduce melalui tahap Map, Shuffle & Sort, dan Reduce.
- Memahami keterkaitan antara HDFS, YARN, dan MapReduce dalam satu klaster yang sama.
- Menyusun dan menjalankan program WordCount menggunakan Java Hadoop.
- Menjalankan job MapReduce pada klaster dengan dua DataNode dan dua NodeManager.
- Mengamati bahwa hasil akhir MapReduce tersimpan kembali ke HDFS.

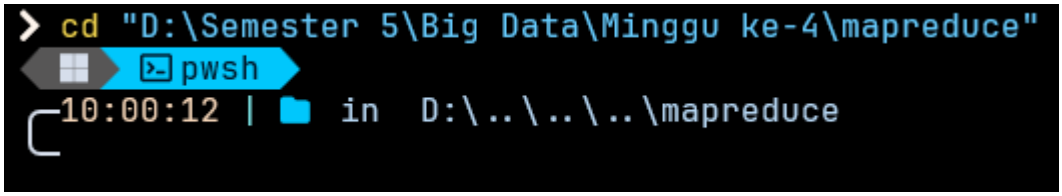
BAB II
DASAR TEORI

2.1 Konsep MapReduce, HDFS, dan YARN

MapReduce lahir dari kebutuhan mengolah data berukuran sangat besar tanpa harus memindahkan seluruh data tersebut menuju satu titik pemrosesan. Pendekatan komputasi tradisional yang bersifat tersentralisasi menghadapi kendala serius ketika volume data mencapai skala terabyte hingga petabyte, sebab jaringan menjadi titik kemacetan dan kapasitas satu server sulit menampung beban tersebut. Google memperkenalkan solusi atas persoalan ini melalui whitepaper GFS dan MapReduce pada tahun 2004, yang kemudian diimplementasikan secara open source oleh Doug Cutting dan Mike Cafarella lewat Apache Hadoop pada 2006. Hadoop menyatukan tiga komponen inti yang saling bergantung, yaitu HDFS sebagai lapisan penyimpanan yang memecah file besar menjadi block dan mereplikasinya ke beberapa DataNode untuk menjaga ketahanan data, YARN sebagai lapisan manajemen sumber daya yang mengalokasikan CPU dan RAM melalui ResourceManager dan NodeManager, serta MapReduce sebagai lapisan komputasi yang mengeksekusi logika pengolahan data. Proses MapReduce sendiri berjalan melalui empat tahap, dimulai dari input split yang memecah dataset menjadi bagian-bagian kecil selaras dengan block HDFS, dilanjutkan fase map yang mengubah setiap baris data menjadi pasangan key-value secara paralel di berbagai node, kemudian fase shuffle and sort yang dikelola sepenuhnya oleh framework untuk mengelompokkan key yang sama, dan diakhiri fase reduce yang melakukan agregasi nilai dari setiap key sebelum hasil akhirnya ditulis kembali ke HDFS. Prinsip data locality menjadi kunci dari seluruh mekanisme ini, karena program yang dijalankan justru didekatkan ke lokasi data tersimpan, sehingga transfer data lewat jaringan dapat ditekan seminimal mungkin dan proses paralel di ribuan node biasa tetap dapat berjalan efisien.

BAB III
PEMBAHASAN

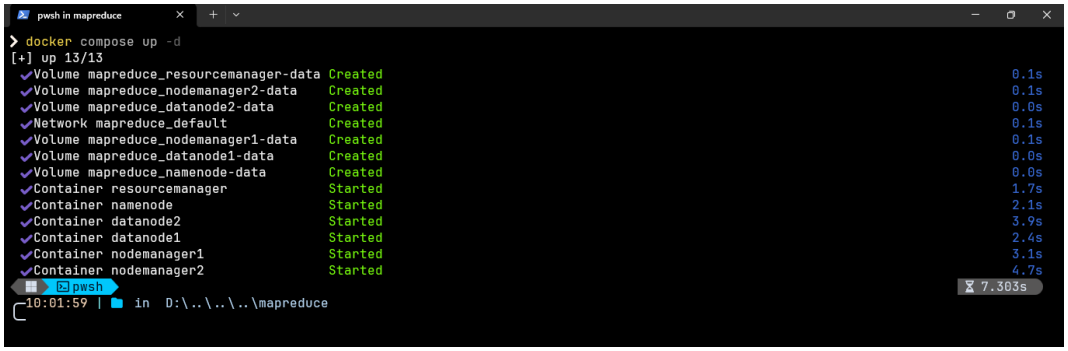
3.1 Hasil dan Tangkapan Layar



Gambar 3.1.1 - Pindah ke direktori tugas Minggu ke-4.png

```
cd "D:\Semester 5\Big Data\Minggu ke-4\mapreduce"
```

Menampilkan perintah cd pada PowerShell untuk berpindah ke folder kerja proyek MapReduce di D:\Semester 5\Big Data\Minggu ke-4\mapreduce, sebagai langkah awal sebelum seluruh proses praktikum dijalankan.



Gambar 3.1.2 - Jalankan klaster Hadoop + YARN.png

```
docker compose up -d
```

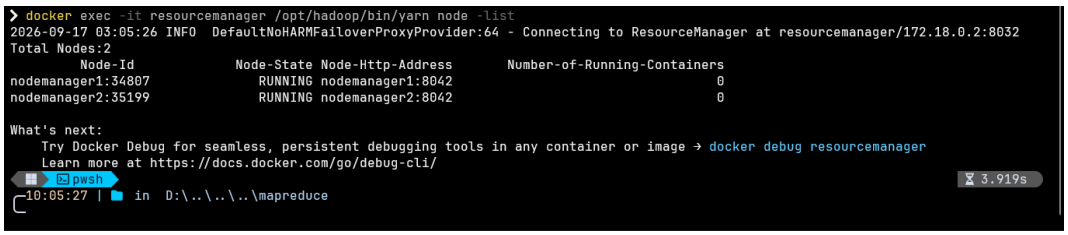
Perintah docker compose up -d dijalankan untuk membangun klaster sesuai konfigurasi pada docker-compose.yml. Output menunjukkan 13 dari 13 objek berhasil dibuat, termasuk enam container utama (namenode, datanode1, datanode2, resourcemanager, nodemanager1, nodemanager2) yang semuanya berstatus Started.



Gambar 3.1.3 - Verifikasi apakah keenam layanan sudah menyala.png

```
docker compose ps -a
```

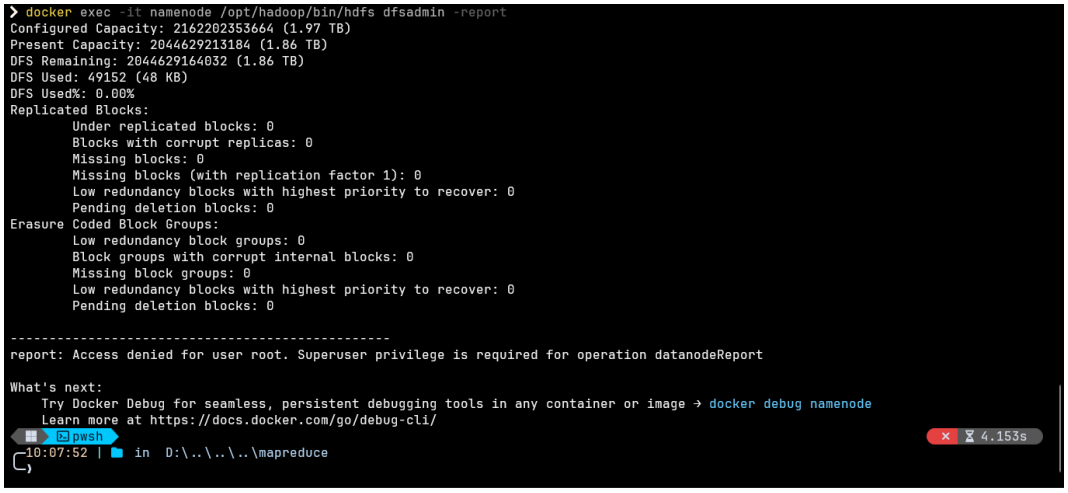
Perintah docker compose ps -a menampilkan status keenam container dalam kondisi Up, lengkap dengan pemetaan port masing-masing layanan: 8020 dan 9870 pada namenode, 8042 pada nodemanager1, 8043 pada nodemanager2, serta 8088 pada resourcemanager.



Gambar 3.1.4 - Verifikasi YARN Node.png

```
docker exec -it resourcemanager /opt/hadoop/bin/yarn node -list
```

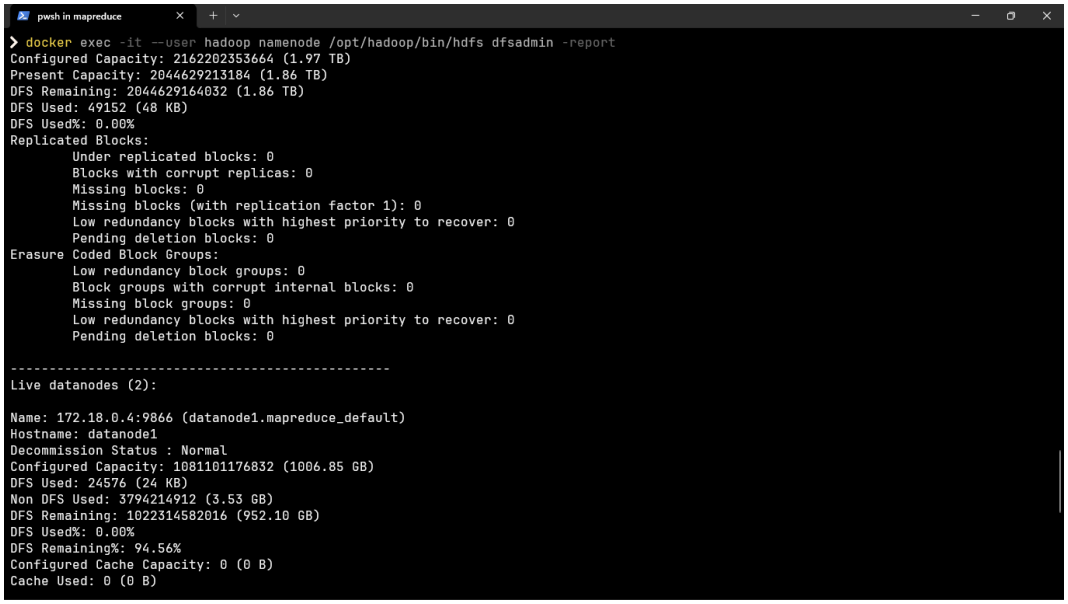
Perintah yarn node -list dijalankan dari dalam container resourcemanager. Hasilnya menunjukkan Total Nodes: 2, dengan nodemanager1 dan nodemanager2 sama-sama berstatus RUNNING, membuktikan YARN sudah mengenali kedua worker node pada klaster.



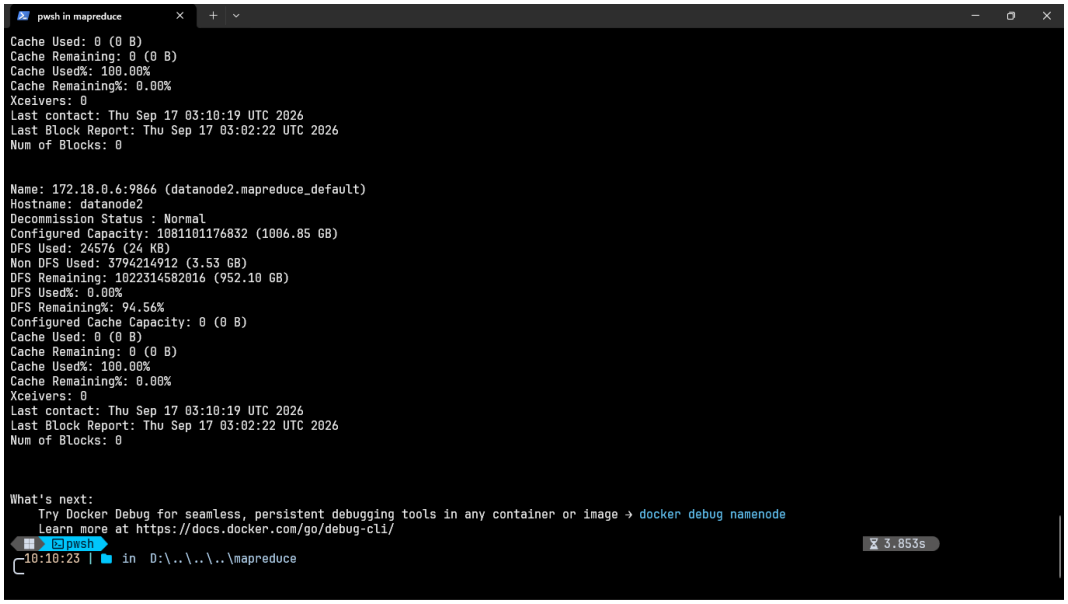
Gambar 3.1.5 - Verifikasi DataNode HDFS - Access denied for user root.png

```
docker exec -it namenode /opt/hadoop/bin/hdfs dfsadmin -report
```

Percobaan pertama menjalankan hdfs dfsadmin -report tanpa menentukan user menghasilkan error "Access denied for user root. Superuser privilege is required for operation datanodeReport". Error ini muncul karena perintah dijalankan sebagai user root, sementara operasi laporan datanode pada HDFS mensyaratkan user superuser, yaitu hadoop.



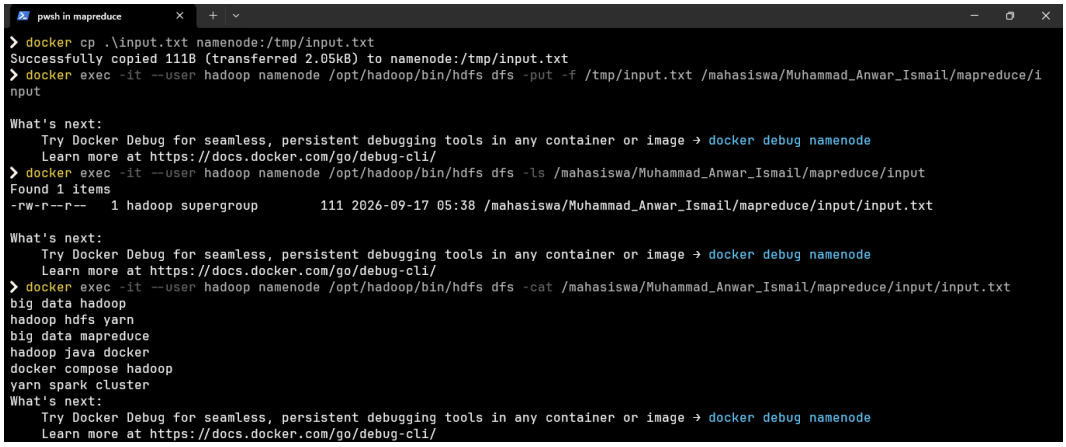
Gambar 3.1.6 - Verifikasi DataNode HDFS_1.png



Gambar 3.1.7 - Verifikasi DataNode HDFS_2.png

```
docker exec -it –user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report
```

Setelah perintah diulang dengan tambahan opsi --user hadoop, laporan HDFS berhasil ditampilkan tanpa error. Kedua gambar ini menunjukkan kapasitas klaster sebesar 1,97 TB serta status "Live datanodes (2)", membuktikan datanode1 dan datanode2 keduanya aktif dan sudah tercatat pada NameNode.



Gambar 3.1.8 - Menyiapkan File Input di HDFS.png

```
docker cp .\input.txt namenode:/tmp/input.txt
```

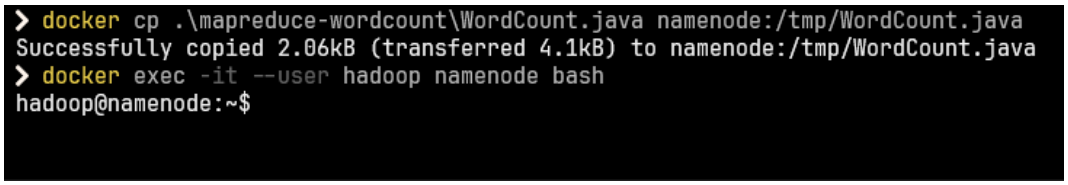
```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -put -f /tmp/input.txt
/mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input/input.txt
```

Rangkaian perintah menyalin input.txt dari komputer lokal ke container namenode, mengunggah ulang ke folder /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input dengan opsi -put -f supaya file lama otomatis tertimpa, lalu menampilkan isinya. File input.txt yang dipakai berisi enam baris kalimat pendek seputar istilah big data dan Hadoop, dengan total sebelas kata unik agar memenuhi syarat minimal sepuluh kata berbeda pada Tugas Praktikum.



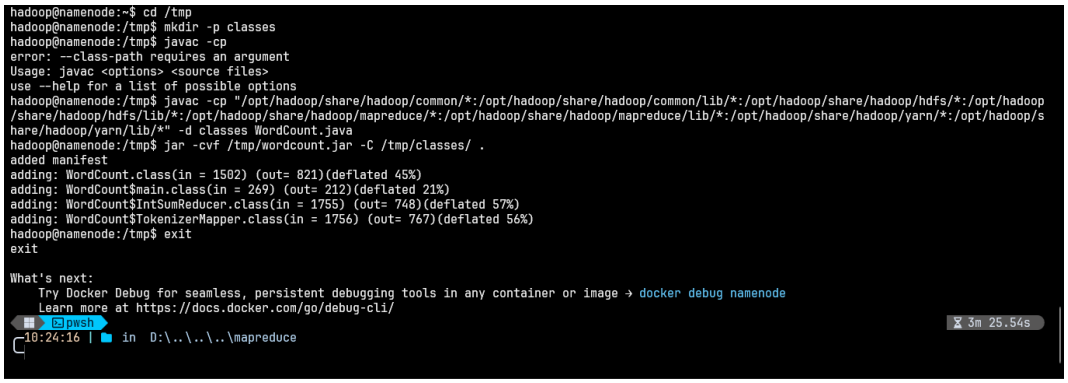
Gambar 3.1.9 - Kompilasi Java (Membuat file JAR).png

```
docker cp .\mapreduce-wordcount\WordCount.java
namenode:/tmp/WordCount.java

docker exec -it --user hadoop namenode bash
```

File WordCount.java disalin ke dalam container namenode menggunakan docker cp, kemudian sesi terminal Linux dibuka dengan docker exec --user

hadoop namenode bash untuk melanjutkan proses kompilasi pada langkah berikutnya.



Gambar 3.1.10 - Rangkaian proses kompilasi Java di dalam terminal Linux.png

```
cd /tmp

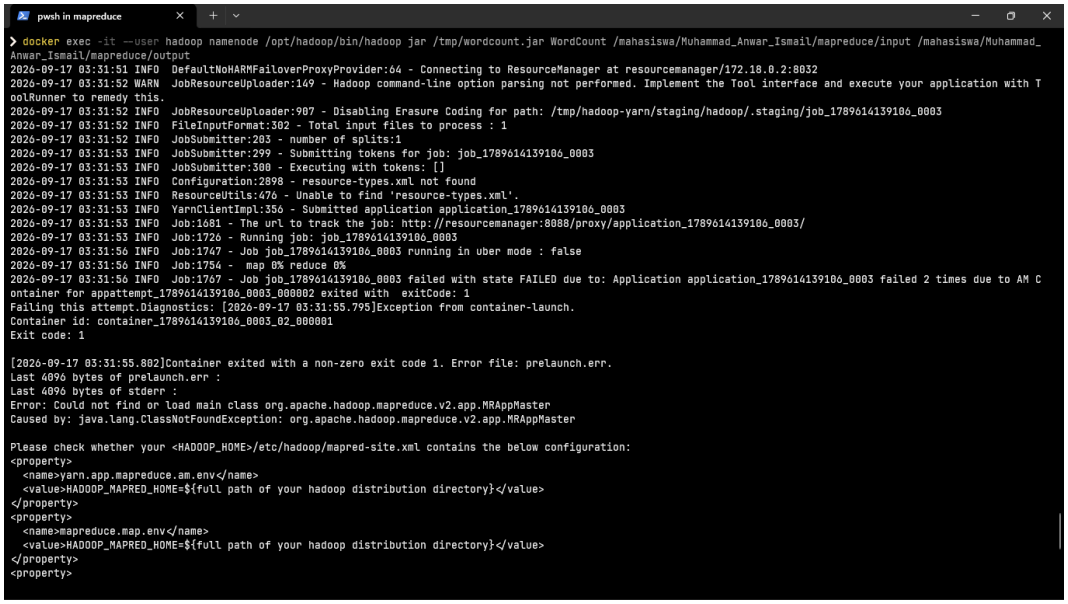
mkdir -p classes

javac -cp "/opt/hadoop/share/hadoop/common/*:/opt/hadoop/share/hadoop/common/lib/*:/opt/hadoop/share/hadoop/hdfs/*:/opt/hadoop/share/hadoop/hdfs/lib/*:/opt/hadoop/share/hadoop/mapreduce/*:/opt/hadoop/share/hadoop/mapreduce/lib/*:/opt/hadoop/share/hadoop/yarn/*:/opt/hadoop/share/hadoop/yarn/lib/*" -d classes WordCount.java

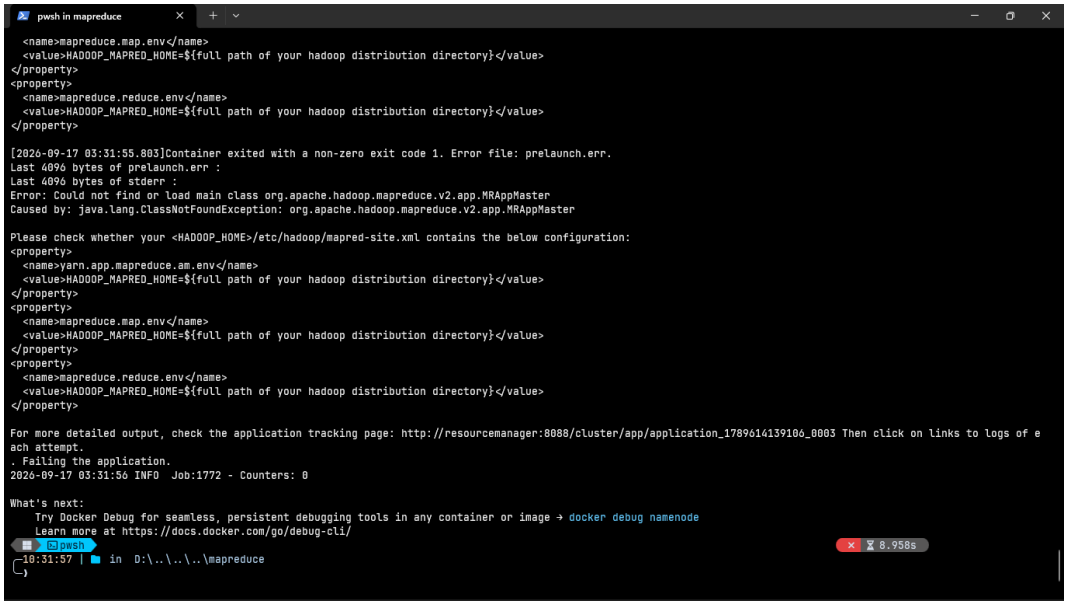
jar -cvf /tmp/wordcount.jar -C /tmp/classes/ .

exit
```

Percobaan pertama kompilasi sempat gagal dengan pesan error "--class-path requires an argument" karena classpath yang panjang tidak dibungkus tanda kutip sehingga terputus oleh shell. Setelah classpath ditulis ulang dengan tanda kutip, perintah javac berhasil dijalankan dan menghasilkan empat file class (WordCount, WordCount$main, IntSumReducer, TokenizerMapper), yang kemudian dikemas menjadi wordcount.jar lewat perintah jar -cvf.



Gambar 3.1.11 - Menjalankan MapReduce Job - Error_1.png



Gambar 3.1.12 - Menjalankan MapReduce Job - Error_2.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output
```

Error ClassNotFoundException pada MRAppMaster Pada saat eksekusi awal perintah *submit job* MapReduce, sistem mendeteksi terjadinya kegagalan operasi

(*Job Failed*) dengan melontarkan eksepsi java.lang ClassNotFoundException: org.apache.hadoop.mapreduce.v2.app.MRAppMaster. Berdasarkan analisis pembacaan log *error* di terminal, anomali ini terjadi karena arsitektur YARN pada *image* Docker Apache Hadoop versi 3.4.3 tidak dapat mendeteksi variabel *environment path* menuju direktori distribusi MapReduce secara otomatis.

Untuk mengatasi kendala teknis tersebut, modifikasi struktural dilakukan pada file mapred-site.xml. Saya menambahkan 3 blok <property> baru di dalam tag <configuration> untuk mendefinisikan lokasi (*path*) MapReduce agar YARN tidak kebingungan mencarinya. Ini kodenya:

```
<property>
  <name>yarn.app.mapreduce.am.env</name>
  <value>HADOOP_MAPRED_HOME=/opt/hadoop</value>
</property>
<property>
  <name>mapreduce.map.env</name>
  <value>HADOOP_MAPRED_HOME=/opt/hadoop</value>
</property>
<property>
  <name>mapreduce.reduce.env</name>
  <value>HADOOP_MAPRED_HOME=/opt/hadoop</value>
</property>
```

Konfigurasi diperbarui dengan menyuntikkan tiga parameter environment variable tambahan secara eksplisit, yakni yarn.app.mapreduce.am.env, mapreduce.map.env, dan mapreduce.reduce.env, di mana ketiganya diarahkan ke nilai absolut direktori instalasi Hadoop (HADOOP_MAPRED_HOME=/opt/hadoop).



Gambar 3.1.13 - 10. Matikan - Hidupkan - Hapus FOlder Output yang gagal.png

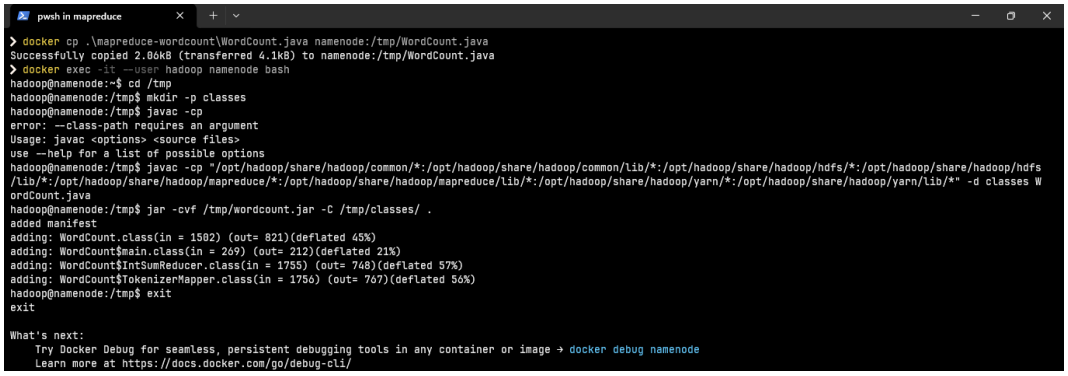
```
docker compose down

docker compose up -d

docker exec -it –user Hadoop namenode /opt/Hadoop/bin/hdfs dfs -rm -r -f /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output

docker exec -it –user Hadoop namenode /opt/Hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output
```

Klaster dimatikan dan dinyalakan ulang lewat docker compose down lalu docker compose up -d agar konfigurasi mapred-site.xml yang sudah diperbarui terbaca dari awal. Folder output dari percobaan gagal sebelumnya turut dihapus dengan hdfs dfs -rm -r -f, dan pengecekan berikutnya mengonfirmasi folder tersebut memang sudah tidak ada.



Gambar 3.1.14 – 11. Kirim ulang kode Java - Masuk ke dalam terminal Linux NameNode - Lakukan kompilasi

```
docker cp .\mapreduce-wordcount\WordCount.java
namenode:/tmp/WordCount.java

docker exec -it --user hadoop namenode bash

cd /tmp

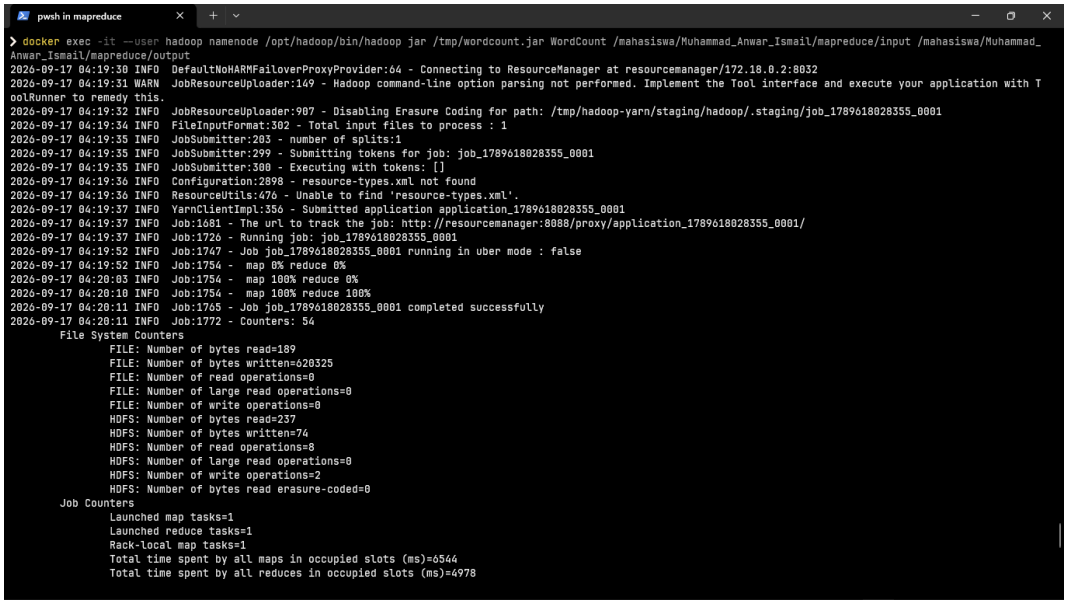
mkdir -p classes

javac -cp
"/opt/hadoop/share/hadoop/common/*:/opt/hadoop/share/hadoop/co
mmon/lib/*:/opt/hadoop/share/hadoop/hdfs/*:/opt/hadoop/share/hado
op/hdfs/lib/*:/opt/hadoop/share/hadoop/mapreduce/*:/opt/hadoop/sh
are/hadoop/mapreduce/lib/*:/opt/hadoop/share/hadoop/yarn/*:/opt/ha
doop/share/hadoop/yarn/lib/*" -d classes WordCount.java

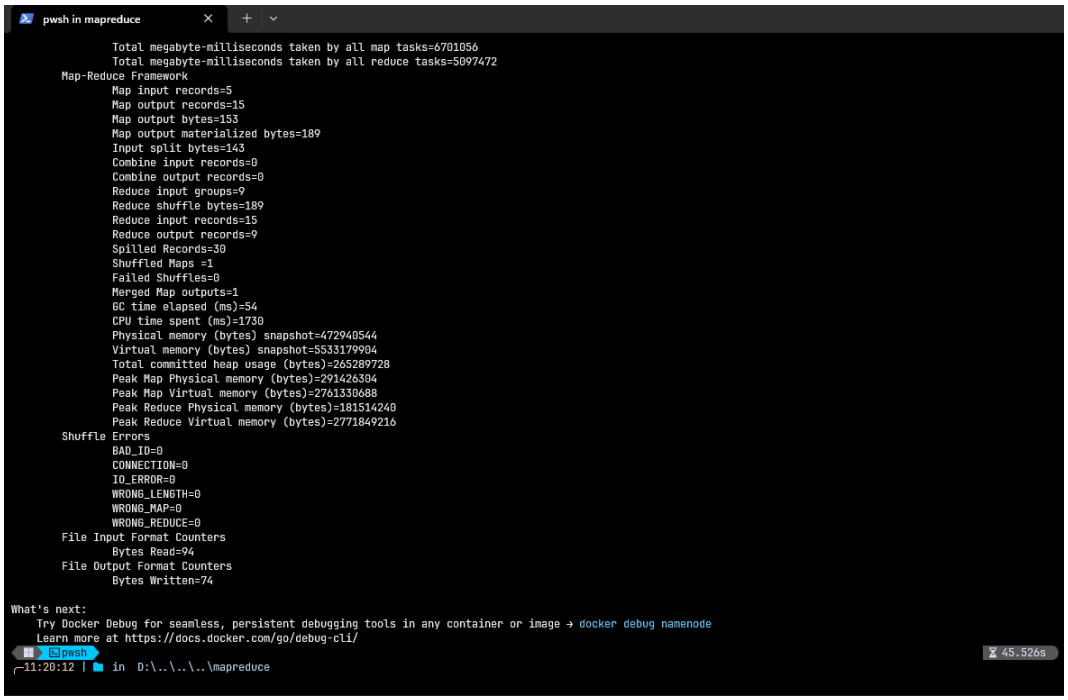
jar -cvf /tmp/wordcount.jar -C /tmp/classes/ .

exit
```

Setelah klaster aktif kembali, WordCount.java dikirim ulang ke container dan proses kompilasi diulang dari awal, mulai dari javac sampai pembuatan wordcount.jar, agar file jar yang dipakai konsisten dengan environment yang baru.



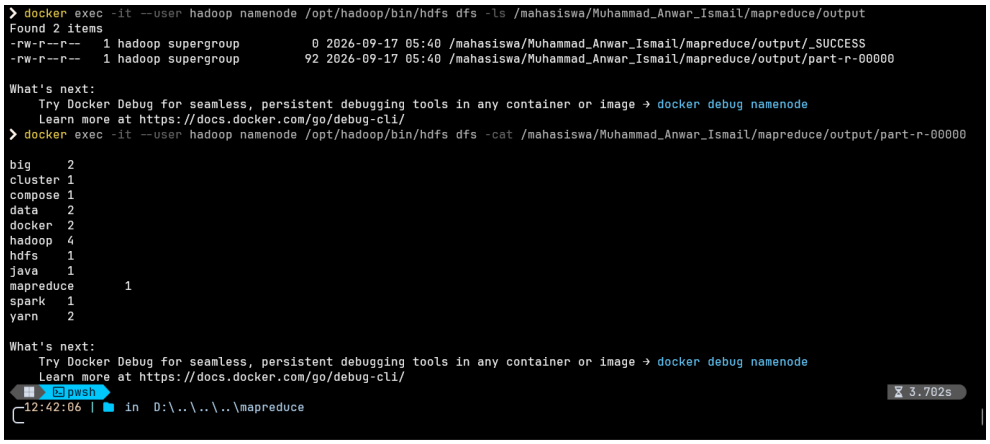
Gambar 3.1.15 - Jalankan Ulang MapReduce Job_1.png



Gambar 3.1.16 - Jalankan Ulang MapReduce Job_2.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output
```

Job dijalankan kembali menggunakan wordcount.jar hasil kompilasi terbaru. Log kali ini menunjukkan progres map 100% dan reduce 100%, diakhiri pesan "Job ... completed successfully", yang berarti penambahan properti pada mapred-site.xml berhasil menyelesaikan error ClassNotFoundException sebelumnya.



Gambar 3.1.17 - Membaca Hasil Akhir dari HDFS

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output

docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output/part-r-00000
```

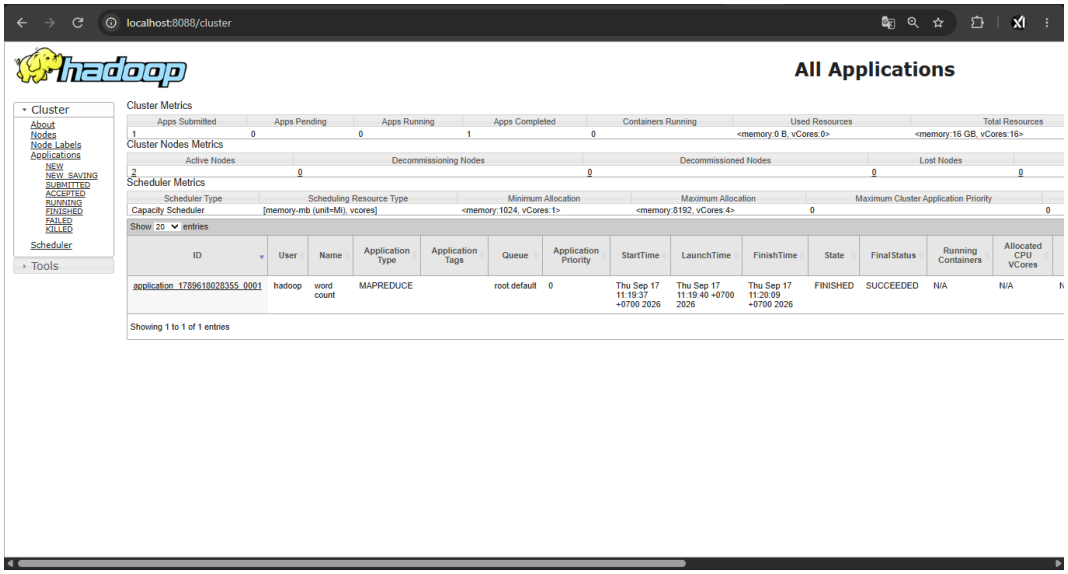
Perintah hdfs dfs -ls menunjukkan folder output berisi dua file, yaitu _SUCCESS dan part-r-00000. Isi part-r-00000 yang dibaca lewat hdfs dfs -cat menampilkan sebelas kata unik beserta jumlah kemunculannya, yaitu hadoop (4), big, data, docker, yarn masing-masing (2), serta cluster, compose, hdfs, java, mapreduce, spark masing-masing (1).



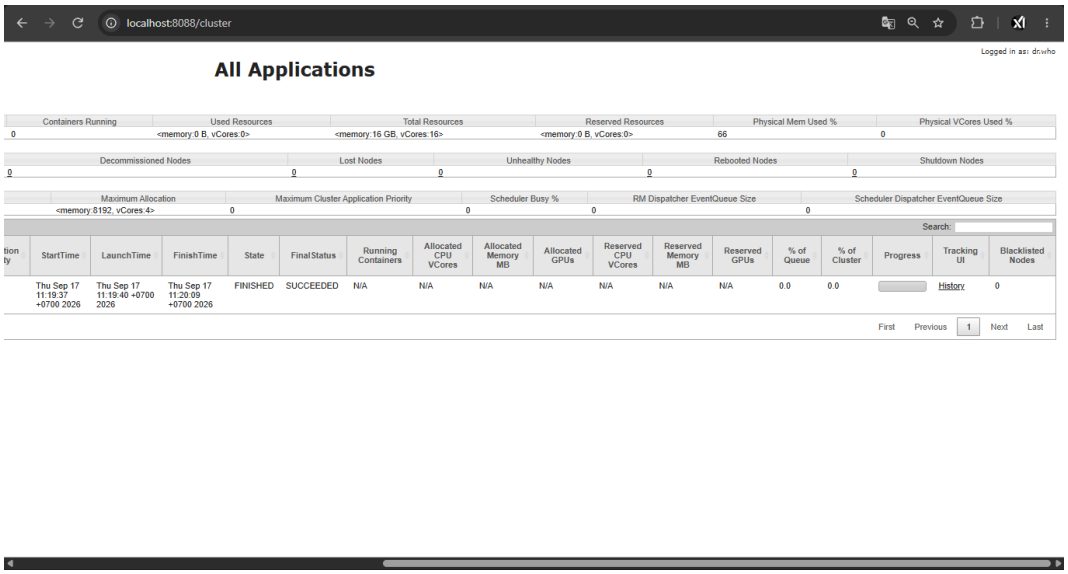
Gambar 3.1.18 - Memeriksa Port YARN Web UI melalui Powershell.png

```
docker compose ps
```

Perintah docker compose ps dijalankan untuk memastikan seluruh container masih berjalan setelah lebih dari 27 menit, sekaligus mengonfirmasi ulang port yang terbuka pada masing-masing layanan sebelum membuka YARN Web UI lewat browser.



Gambar 3.1.19 - Memeriksa YARN Web UI_1.png



Gambar 3.1.20 - Memeriksa YARN Web UI_1.png

Halaman http://localhost:8088/cluster menampilkan satu aplikasi bertipe MAPREDUCE dengan nama "word count", berstatus FINISHED dengan Final Status SUCCEEDED. Detail cluster metrics juga memperlihatkan dua Active Nodes serta total resource 16 GB memori dan 16 vCores, menandakan kedua NodeManager memang ikut menyumbang kapasitas pemrosesan pada klaster.

3.2 Penelusuran Proses Map ke Shuffle dan Sort ke Reduce

Input.txt yang dipakai pada praktikum ini berisi enam baris dengan total delapan belas kata, yaitu "big data hadoop", "hadoop hdfs yarn", "big data mapreduce", "hadoop java docker", "docker compose hadoop", dan "yarn spark cluster". Pada fase map, TokenizerMapper membaca file tersebut baris demi baris dan memecah setiap baris menjadi token, lalu setiap token diberi nilai satu, sehingga baris pertama menghasilkan tiga pasangan (big,1), (data,1), (hadoop,1), baris kedua menghasilkan (hadoop,1), (hdfs,1), (yarn,1), dan pola yang sama berlanjut hingga baris keenam. Keseluruhan proses ini menghasilkan delapan belas pasangan key-value yang tersebar di berbagai node secara paralel, sesuai jumlah kata pada file input.

Fase shuffle and sort kemudian mengambil alih tanpa campur tangan kode yang ditulis sendiri. Framework Hadoop mengelompokkan seluruh pasangan berdasarkan key yang sama, sehingga kata hadoop yang muncul di empat baris berbeda dikumpulkan menjadi satu grup dengan daftar nilai [1,1,1,1], kata big menjadi [1,1], dan kata-kata yang hanya muncul sekali seperti cluster, compose, hdfs, java, mapreduce, serta spark masing-masing menjadi grup dengan satu nilai saja. Proses ini menghasilkan sebelas grup key yang berbeda, sesuai jumlah kata unik pada input.txt.

Pada fase reduce, IntSumReducer menerima setiap grup tersebut dan menjumlahkan seluruh nilai di dalamnya. Grup hadoop dengan daftar [1,1,1,1] dijumlahkan menjadi 4, grup big, data, docker, dan yarn yang masing-masing berisi [1,1] dijumlahkan menjadi 2, sedangkan grup cluster, compose, hdfs, java, mapreduce, dan spark yang hanya berisi satu nilai tetap menghasilkan 1. Hasil

penjumlahan inilah yang kemudian ditulis ke HDFS dan terlihat pada Gambar 3.1.17.

3.3 Perbedaan Komponen dalam Klaster

Komponen	Peran dalam Klaster
NameNode	Mengelola metadata HDFS, termasuk struktur direktori dan lokasi block, tanpa menyimpan data mentah secara langsung.
DataNode	Menyimpan block data fisik hasil pemecahan file oleh HDFS, dan melapor secara berkala ke NameNode.
ResourceManager	Mengelola alokasi resource CPU dan RAM di seluruh klaster serta menjadwalkan aplikasi yang masuk melalui YARN.
NodeManager	Berjalan di setiap worker node, menjalankan dan memantau container/task yang dialokasikan ResourceManager.
Mapper	Kode yang ditulis pada TokenizerMapper, bertugas memecah setiap baris input menjadi pasangan key-value awal.
Reducer	Kode yang ditulis pada IntSumReducer, bertugas menjumlahkan seluruh nilai dari key yang sama menjadi hasil akhir.
Driver	Kelas main() pada WordCount.java yang mengonfigurasi job,

	menghubungkan Mapper dan Reducer, lalu mensubmit job ke YARN.

Ketujuh komponen ini saling melengkapi tapi menjalankan peran yang berbeda, NameNode dan DataNode menangani penyimpanan lewat HDFS, ResourceManager dan NodeManager menangani alokasi resource lewat YARN, sedangkan Mapper, Reducer, dan Driver adalah bagian dari logika program MapReduce yang ditulis sendiri oleh mahasiswa.

BAB IV
KESIMPULAN

4.1 Kesimpulan

Praktikum ini berhasil membangun klaster Hadoop 3.4.3 menggunakan Docker dengan konfigurasi satu NameNode, dua DataNode, satu ResourceManager, dan dua NodeManager, yang seluruhnya terbukti aktif lewat hasil `hdfs dfsadmin -report` dan `yarn node -list`. Proses menjalankan job WordCount sempat terkendala error ClassNotFoundException pada MRAppMaster akibat YARN belum menemukan environment path menuju direktori distribusi MapReduce, dan kendala ini berhasil diatasi dengan menambahkan tiga properti environment variable pada mapred-site.xml. Setelah perbaikan tersebut, job berjalan sampai selesai dan menghasilkan sebelas kata unik dari input.txt yang berisi delapan belas kata, sesuai dengan jumlah minimal kata berbeda yang diminta pada Tugas Praktikum. Keseluruhan proses ini menunjukkan bahwa hasil akhir MapReduce memang tersimpan kembali ke HDFS melalui kerja sama NameNode, DataNode, ResourceManager, dan NodeManager, sedangkan Mapper, Reducer, dan Driver menjalankan logika pengolahan data yang sudah ditulis sendiri. Kendala teknis berupa error ClassNotFoundException menjadi bagian paling berkesan dari praktikum ini, karena penyelesaiannya membutuhkan pembacaan log error secara teliti dan pemahaman tentang bagaimana YARN mencari environment path saat menjalankan aplikasi MapReduce.

LAMPIRAN A
COMMAND UTAMA

No.	Tahapan	Perintah
1.	Pindah ke folder kerja	cd "D:\Semester 5\Big Data\Minggu ke-4\mapreduce"
2.	Menjalankan klaster	docker compose up -d
3.	Verifikasi status container	docker compose ps -a
4.	Verifikasi YARN node	docker exec -it resourcemanager /opt/hadoop/bin/yarn node -list
5.	Verifikasi DataNode (percobaan pertama, gagal)	docker exec -it namenode /opt/hadoop/bin/hdfs dfsadmin -report
6.	Verifikasi DataNode (dengan user hadoop, berhasil)	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report
7.	Menyalin input.txt ke container	docker cp .\input.txt namenode:/tmp/input.txt
8.	Membuat folder input di HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input
9.	Mengunggah input.txt ke HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -put -f /tmp/input.txt

		/mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input
10.	Memeriksa daftar file input	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input
11.	Membaca isi input.txt di HDFS	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input/input.txt
12.	Menyalin WordCount.java ke container	docker cp .\mapreduce-wordcount\WordCount.java namenode:/tmp/WordCount.java
13.	Masuk ke terminal Linux container	docker exec -it --user hadoop namenode bash
14.	Kompilasi Java menjadi class	javac -cp "/opt/hadoop/share/hadoop/common/*:...:/opt/hadoop/share/hadoop/yarn/lib/*" -d classes WordCount.java
15.	Membuat file JAR	jar -cvf /tmp/wordcount.jar -C /tmp/classes/ .
16.	Menjalankan job (percobaan pertama, gagal)	docker exec -it --user hadoop namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input

		/mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output
17.	Restart klaster setelah update mapred-site.xml	docker compose down lalu docker compose up -d
18.	Menghapus folder output yang gagal	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -rm -r -f /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output
19.	Mengulang kompilasi Java	Ulangi langkah 12–15
20.	Menjalankan job (percobaan kedua, berhasil)	docker exec -it --user hadoop namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/input /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output
21.	Memeriksa daftar file output	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output
22.	Membaca hasil akhir WordCount	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /mahasiswa/Muhammad_Anwar_Ismail/mapreduce/output/part-r-00000

23.	Memeriksa status container sebelum membuka YARN UI	docker compose ps

LAMPIRAN B

PORT YANG DIGUNAKAN

Layanan	Port Host dan Container	Fungsi
NameNode (HDFS Web UI)	9870	Mengakses HDFS Web UI lewat browser
NameNode (RPC)	8020	Komunikasi internal client dengan HDFS
ResourceManager (YARN Web UI)	8088	Mengakses YARN Web UI dan memantau job MapReduce
NodeManager 1	8042	Web UI NodeManager 1
NodeManager 2	8043	Web UI NodeManager 2 (port host digeser ke 8043 supaya tidak bentrok dengan NodeManager 1)